Project Report:

Robot Car Contest

จัดทำโดย

นาย จิรัฏฐ์ มหาธนเศรษฐ์ 68010159

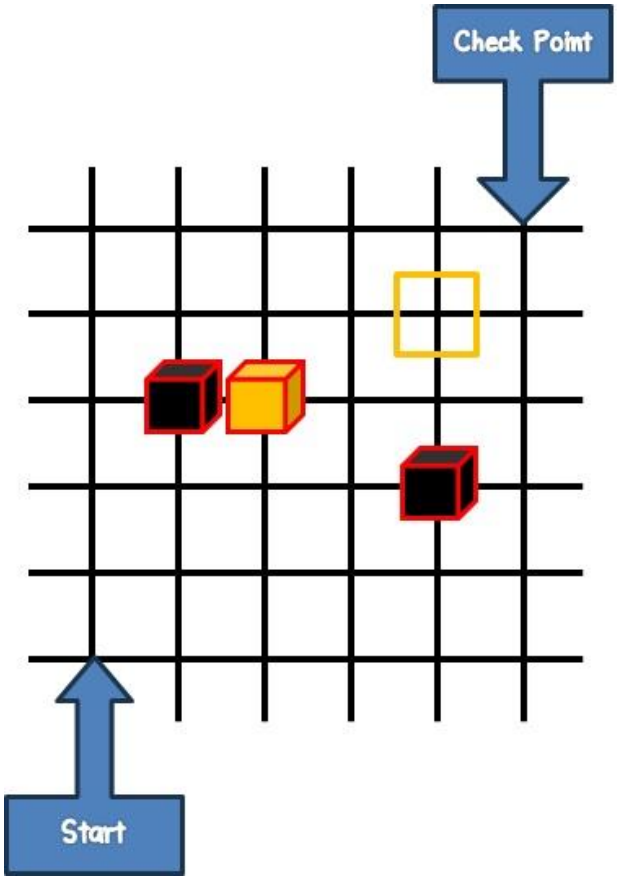
นาย ธนวิชญ์ ลิ้มศิริเศรษฐกุล 68010483

Concept:

หลักการทำงานของ Robot Car

หุ่นยนต์ถูกออกแบบให้สามารถเคลื่อนที่อัตโนมัติตามเส้นสีดำบนพื้นสีขาว โดยสนามทดสอบมีลักษณะเป็นตารางขนาด 6x6 ช่อง (ประมาณ 4 ตารางเมตร) แต่ละช่องมีขนาด 30x30 ซม. และเส้นนำทางมีความหนาประมาณ 2–4 ซม. จุดเริ่มต้นอยู่ที่มุมล่างซ้าย และจุดสิ้นสุดอยู่ที่มุมบนขวา โดยมีภารกิจหลักคือ:

- ตรวจจับสิ่งกีดขวาง (กล่องสีดำ) และหลีกเลี่ยง
- เคลื่อนที่ไปยังตำแหน่งกล่องสีส้ม
- ผลักกล่องสีส้มไปยังจุดเป้าหมาย
- เดินทางไปยังจุดสิ้นสุด
- กลับสู่จุดเริ่มต้น



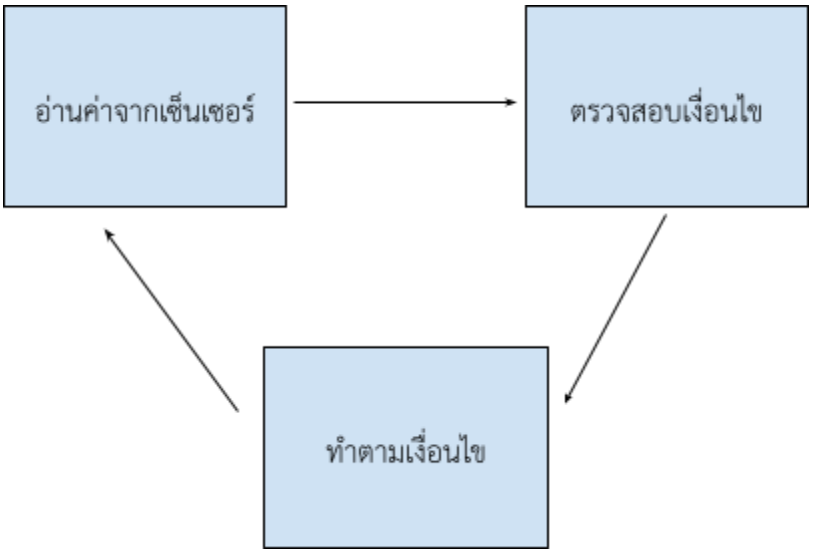
การเดินตามเส้นด้วย PID

หุ่นยนต์ใช้เซ็นเซอร์ TCRT5000 จำนวน 5 ตัวในการตรวจจับเส้นสีดำ โดยใช้หลักการ PID (Proportional–Integral–Derivative) เพื่อปรับความเร็วของมอเตอร์ซ้ายและขวาให้เหมาะสมกับตำแหน่งของเส้นที่ตรวจพบ

ค่าที่เซ็นเซอร์อ่านได้	การเคลื่อนที่	มอเตอร์ซ้าย	มอเตอร์ขวา
11011	เดินตรง	✓	✓
01111 / 10111	เลี้ยวซ้าย	✗	✓
11110 / 11101	เลี้ยวขวา	✓	✗
00000	หยุด	✗	✗
11111	หลุดเส้น	ถอยหลัง	ถอยหลัง

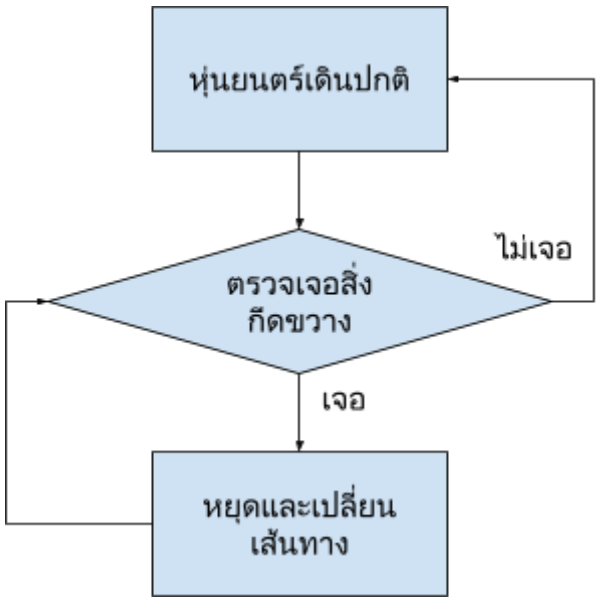
หมายเหตุ: 1 คือ เจอสีขาว, 0 คือ เจอสีดำ

โดยที่เราจะทำการเก็บค่าที่ sensor อ่านค่าได้ แปลงเป็น Binary และนำเลขเหล่านั้นไปทำเงื่อนไขของหุ่นยนต์ เช่น 00100 เมื่อเข้าเงื่อนไข จะทำการสั่งให้หุ่นยนต์ เดินตรง และจะทำเช่นเดียวกันกับค่าอื่น ๆ ที่รับมาได้ เพื่อรองรับเงื่อนไขทุกกรณีที่เป็นไปได้ในแต่ละโค้ง



การจัดการกับสิ่งกีดขวาง

หุ่นยนต์ใช้ Ultrasonic sensor ในการตรวจจับสิ่งกีดขวางด้านหน้า หากพบสิ่งกีดขวางในระยะน้อยกว่า 25 ซม. จะหยุดและบันทึกตำแหน่งกล่องดำลงในแผนที่ จากนั้นจะเลือกเส้นทางใหม่โดยหลีกเลี่ยงกล่องดำที่ตรวจพบ



การจัดการกับกล่องสีส้ม

เมื่อหุ่นยนต์เดินทางไปถึงตำแหน่งกล่องสีส้ม พิกัด 2,3 จะทำการผลักกล่องไปยังตำแหน่งเป้าหมาย พิกัด 4,4 โดยใช้การเดินตามเส้นและการตรวจจับทางแยกเพื่อปรับทิศทางให้เหมาะสม

การจัดการกับทางแยก

หุ่นยนต์จะตรวจจับทางแยกโดยใช้เงื่อนไขว่าเซ็นเซอร์TCRT5000 ตรวจพบเส้นสีดำ 5 จุด เมื่อถึงทางแยกจะหยุดชั่วคราว อัปเดตพิกัด และเลือกทิศทางใหม่โดยพิจารณาจากตำแหน่งเป้าหมายและกล่องดำที่ตรวจพบ

การกลับสู่จุดเริ่มต้น

หลังจากผลักกล่องสีส้มและเดินทางไปยังจุดสิ้นสุด (5,5) หุ่นยนต์จะกลับสู่จุดเริ่มต้น (0,0) โดยใช้แผนที่ที่สร้างขึ้นระหว่างการเดินทางเพื่อหลีกเลี่ยงสิ่งกีดขวา

การออกแบบ:

Sensor ที่ใช้

- TCRT5000 Sensor (5 ตัว): ตรวจจับเส้นสีดำ
- Ultrasonic Sensor (1 ตัว): ตรวจจับสิ่งกีดขวาง
- Buzzer: แจ้งเตือนเมื่อพบสิ่งกีดขวาง

Algorithm ที่ใช้

- PID Line Following
- Grid Mapping
- Obstacle Avoidance
- Orange Box Pushing Logic

Hardware:

---

1. Arduino UNO R3
2. Motor Driver L298N
3. มอเตอร์ล้อ 2 ตัว
4. Sensor TCRT5000 จำนวน 5 ตัว
5. Ultrasonic Sensor 1 ตัว
6. Buzzer 1 ตัว
7. โครงรถหุ่นยนต์
8. สายไฟ, รางถ่าน, แบตเตอรี่ 2 ก้อน

ปัญหาและการแก้ไข:

ปัญหาด้าน Software

- การคำนวณ PID ต้องปรับค่า Kp, Ki, Kd ให้เหมาะสมกับความเร็วและน้ำหนักของรถ
- การอัปเดตพิกัดบางครั้งคลาดเคลื่อน ต้องเพิ่มการตรวจสอบทางแยกให้แม่นยำ

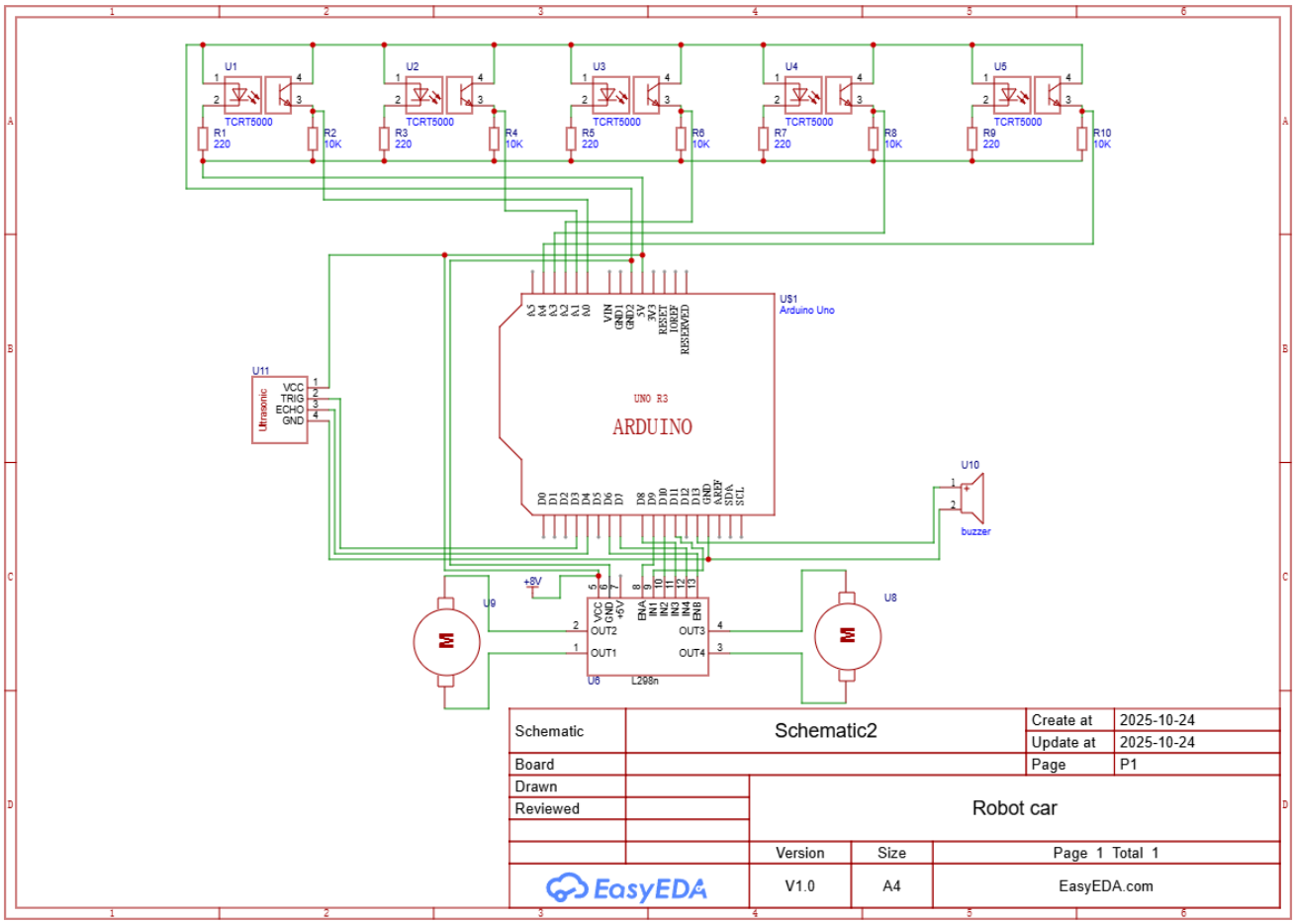
ปัญหาด้าน Hardware

- เซ็นเซอร์บางตัวมีความไวต่ำ ต้องเปลี่ยนใหม่
- มอเตอร์มีแรงไม่พอในการผลักกล่องสีส้ม ต้องเพิ่มแรงดันไฟฟ้า
- สนามไม่เรียบ ทำให้เซ็นเซอร์อ่านค่าผิดพลาด ต้องปรับพื้นสนาม

สรุปผลการทำงาน:

หุ่นยนต์สามารถเดินตามเส้น ตรวจจับสิ่งกีดขวาง ผลักกล่องสีส้ม และกลับสู่จุดเริ่มต้นได้สำเร็จตามภารกิจที่กำหนด โดยใช้การประมวลผลแบบ PID และการวางแผนเส้นทางแบบ Grid Mapping ร่วมกับการตรวจจับสิ่งกีดขวางแบบเรียลไทม์

Circuit Diagram:



Source Code:

```
// ====== 5-Sensor PID Line Follower + Grid Mapping
+ Orange Box Logic ======
// --- Sensor pins ---
const int S1 = A0; // Leftmost
const int S2 = A1;
const int S3 = A2;
const int S4 = A3;
const int S5 = A4; // Rightmost

// --- Motor pins ---
const int enA = 6; //ซ้าย
const int in1 = 7; //ซ้ายหลัง
const int in2 = 8; //ซ้ายหน้า
const int enB = 9; //ขวา
const int in3 = 10; //ขวาหน้า
const int in4 = 11; //ขวาหลัง

// --- Ultrasonic ---
const int trigPin = 3;
const int echoPin = 4;

// --- Buzzer ---
const int buzzerPin = 13;

// --- Base speed ---
int baseSpeedLeft = 120;
```

```
int baseSpeedRight = 120;

// --- PID parameters ---
float Kp = 45;
float Ki = 0.0;
float Kd = 45;
float error = 0, lastError = 0, integral = 0;

// --- Coordinate system ---
int x = 0, y = -1;
int direction = 0; // 0=N,1=E,2=S,3=W

// --- Intersection flags ---
bool lastIntersection = false;
bool firstIntersection = true;

// --- Orange Box ---
// --- Black Boxes on edge ---
bool blackBoxes[6][6] = {false};

// --- Target coordinate ---
int targetX = 1, targetY = 1;

// ================== FUNCTION PROTOTYPES
==================
void setMotor(int leftSpeed,int rightSpeed);
void stopMotors();
void rotateTo(int newDir);
```

```
void rotateLeft();
void rotateRight();
void turnAround();
void updatePosition();
void advancePastIntersection();
bool atIntersection();
void lineFollowPID();
void chooseDirection();
long readUltrasonic();
void printCurrentPosition();
void playsound();
void soundatposition();
void executePush();
void ReverseFollowPID();
void setReverseMotor(int leftSpeed, int
rightSpeed);

// ================== SETUP ==================
void setup() {
Serial.begin(9600);
pinMode(S1, INPUT);
pinMode(S2, INPUT);
pinMode(S3, INPUT);
pinMode(S4, INPUT);
pinMode(S5, INPUT);

pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
```

```
pinMode(enA, OUTPUT);
pinMode(in1, OUTPUT);
pinMode(in2, OUTPUT);
pinMode(enB, OUTPUT);
pinMode(in3, OUTPUT);
pinMode(in4, OUTPUT);

pinMode(buzzerPin, OUTPUT);
//กล่องดำใน
blackBoxes[1][3] = true;
blackBoxes[4][2] = true;
blackBoxes[2][3] = true;
Serial.println("=== Start 5-Sensor Grid Mapping +
Orange Box Task ===");
}

// ================== LOOP ==================
void loop() {
  // scanAndMoveTo(3,1);
  // scanAndMoveTo(3,4);
  // scanAndMoveTo(2,4);
  // advancePastIntersection();
  // rotateTo(3);
  // executePush();
```

```
// scanAndMoveTo(3,3);
// scanAndMoveTo(3,1);
// scanAndMoveTo(1,1);
// scanAndMoveTo(1,2);
// advancePastIntersection();
// rotateTo(1);
// executePush();



// scanAndMoveTo(2,1);
// scanAndMoveTo(3,1);

// advancePastIntersection();
// rotateTo(3);
// executePush();
// executePush();



// scanAndMoveTo(2,3);
// scanAndMoveTo(2,4);
// advancePastIntersection();
// rotateTo(1);
// executePush();
// blackBoxes[4][4] = true;
```

```
  scanAndMoveTo(5,5);
  playsound(); playsound();
  delay(500);
  x=5,y=5;
  chooseDirection();
  scanAndMoveTo(0,0);
  playsound(); playsound(); playsound();
while(1);
}

void scanAndMoveTo(int tx, int ty) {
targetX = tx; targetY = ty;

if (atIntersection()) {
Serial.println("Starting on an intersection.
Orienting towards new target...");
chooseDirection();
}

while (x != targetX || y != targetY) {
lineFollowPID();

if (atIntersection() && !lastIntersection) {
 stopMotors();
 delay(200);
```

```
updatePosition();
printCurrentPosition();


if (x == targetX && y == targetY) {
    stopMotors();
    Serial.println("Arrived at target point!");
    return;
}

advancePastIntersection();

lastIntersection = true;


bool pathIsClear = false;
while (!pathIsClear) {
chooseDirection();

long dist = readUltrasonic();
delay(50);

if (dist > 0 && dist < 25) {
 pathIsClear = false;
 playsound();

 // Mark the obstacle in the direction we are
currently facing
```

```
  int bx = x, by = y;
  switch (direction) {
  case 0: by++; break; // North
  case 1: bx++; break; // East
  case 2: by--; break; // South
  case 3: bx--; break; // West
  }
  if (!blackBoxes[bx][by]) {
  blackBoxes[bx][by] = true;
  Serial.print("Path blocked. New Black Box at:
(");
  Serial.print(bx); Serial.print(",");
Serial.print(by); Serial.println(")");
  }

 } else {
  pathIsClear = true;
  Serial.println("Path is clear. Proceeding.");
 }
 }

}
else {
 lastIntersection = false;
}
}
}
```

```
// ================== ORANGE BOX PUSH
==================
void executePush() {
  Serial.println("\n--- Executing Sensor-Based Push
Sequence ---");

  // Step 1: ตรวจสอบว่ามีกล่องอยู่ข้างหน้าจริงหรือไม่
  long dist = readUltrasonic();
  if (dist > 0 && dist < 25) {
    Serial.println("Orange box detected. Starting
push.");
    playsound();
  } else {
    Serial.println("No box detected! Aborting push
sequence.");
    return;
  }

  // Step 2: ดันกล่องโดยวิ่งตามเส้นจนเจอสี่แยกถัดไป
  while (true) {
    lineFollowPID();
    if (atIntersection()) {
      updatePosition();
      break;
    }
  }
```

```
  // Step 3: ดันต่อไปอีกเล็กน้อย (Nudge) เพื่อให้ "กล่อง" ไปอยู่กลางสี่แยกข้างหน้า
  // *** TUNE THIS VALUE CAREFULLY! ***
  setMotor(120, 120);
  delay(700);
  stopMotors();

  // --- Step 5: ถอยหลังกลับเข้าที่ด้วย Reverse PID ---
  while (true) {
    ReverseFollowPID(); // วิ่งตามเส้นถอยหลังอย่างต่อเนื่อง
    if (atIntersection()) {
      stopMotors();
      advancePastIntersection();
      delay(200); // หยุดเล็กน้อยเพื่อความเสถียร
      break;
    }
  }
}
// ================== PID ==================
void lineFollowPID(){
int s[5];
s[0]=!digitalRead(S1); s[1]=!digitalRead(S2);
s[2]=!digitalRead(S3);
s[3]=!digitalRead(S4); s[4]=!digitalRead(S5);
int total = s[0]+s[1]+s[2]+s[3]+s[4];
if(total == 0) return;
int weights[5]={-2,-1,0,1,2};
int weightedSum=0;
```

```
for(int i=0;i<5;i++){
 weightedSum += s[i]*weights[i];
}
float position = (float)weightedSum/total;
error = 0-position;
integral += error;
float derivative = error-lastError;
float output = Kp*error + Ki*integral +
Kd*derivative;
lastError = error;
int leftSpeed =
constrain(baseSpeedLeft-output,0,255);
int rightSpeed =
constrain(baseSpeedRight+output,0,255);
setMotor(leftSpeed,rightSpeed);
}

void ReverseFollowPID(){
int s[5];
s[0]=!digitalRead(S1); s[1]=!digitalRead(S2);
s[2]=!digitalRead(S3);
s[3]=!digitalRead(S4); s[4]=!digitalRead(S5);
int total = s[0]+s[1]+s[2]+s[3]+s[4];
 if (total == 0) {
    stopMotors();
    return; // Exit the function after setting the
corrective turn
  }
```

```
int weights[5]={-2,-1,0,1,2};
int weightedSum=0;
for(int i=0;i<5;i++){
 weightedSum += s[i]*weights[i];
}
float position = (float)weightedSum/total;
error = 0-position;
integral += error;
float derivative = error-lastError;
float output = Kp*error + Ki*integral +
Kd*derivative;
lastError = error;
int leftSpeed =
constrain(baseSpeedLeft-20+output,0,255);
int rightSpeed =
constrain(baseSpeedRight-20-output,0,255);
setReverseMotor(leftSpeed,rightSpeed);
}

// ================== INTERSECTION
==================
bool atIntersection(){
int s[5];
s[0]=!digitalRead(S1); s[1]=!digitalRead(S2);
s[2]=!digitalRead(S3);
s[3]=!digitalRead(S4); s[4]=!digitalRead(S5);
return (s[0]+s[1]+s[2]+s[3]+s[4])>=4 ;
}
```

```
void updatePosition() {
  switch (direction) {
    case 0: if (y < 5) y++; break; // North
    case 1: if (x < 5) x++; break; // East
    case 2: if (y > 0) y--; break; // South
    case 3: if (x > 0) x--; break; // West
  }
}

// ================== MOVEMENT ==================
void setMotor(int leftSpeed,int rightSpeed){
analogWrite(enA,leftSpeed); digitalWrite(in1,LOW);
digitalWrite(in2,HIGH);
analogWrite(enB,rightSpeed);
digitalWrite(in3,HIGH); digitalWrite(in4,LOW);
}

void stopMotors(){
analogWrite(enA,0); analogWrite(enB,0);
digitalWrite(in1,LOW); digitalWrite(in2,LOW);
digitalWrite(in3,LOW); digitalWrite(in4,LOW);
delay(500);
}
void setReverseMotor(int leftSpeed,int rightSpeed){
analogWrite(enA,leftSpeed); digitalWrite(in1,HIGH);
digitalWrite(in2,LOW);
```

```
analogWrite(enB,rightSpeed); digitalWrite(in3,LOW);
digitalWrite(in4,HIGH);
}
// ================== ROTATE ==================
void rotateTo(int newDir){
int oldDirection = direction;
int diff = (newDir - direction + 4) % 4;
if (diff == 0) return; // Don't turn if already
facing the right way
if (diff == 1) rotateRight();
else if (diff == 2) turnAround();
else if (diff == 3) rotateLeft();
direction = newDir;
Serial.print("Rotated from ");
Serial.print(oldDirection);
Serial.print(" to ");
Serial.println(direction);
}

void rotateLeft(){
analogWrite(enA,105); analogWrite(enB,97);
digitalWrite(in1,HIGH); digitalWrite(in2,LOW);
digitalWrite(in3,HIGH); digitalWrite(in4,LOW);
delay(600);
stopMotors();
}

void rotateRight(){
```

```
analogWrite(enA,105); analogWrite(enB,97);
digitalWrite(in1,LOW); digitalWrite(in2,HIGH);
digitalWrite(in3,LOW); digitalWrite(in4,HIGH);
delay(600);
stopMotors();
}

void turnAround(){
analogWrite(enA,105); analogWrite(enB,97);
digitalWrite(in1,LOW); digitalWrite(in2,HIGH);
digitalWrite(in3,LOW); digitalWrite(in4,HIGH);
delay(1050); // Tuned for a 180-degree turn
stopMotors();
}

// ================== CHOOSE DIRECTION
==================
void chooseDirection(){
// Logic: Choose the best direction towards the
target without hitting a black box.

// Priority 1: Try to move towards the target X
coordinate
if(x < targetX && (x+1) <= 5 &&
!blackBoxes[x+1][y]) { rotateTo(1); return; } //
East
```

```
if(x > targetX && (x-1) >= 0 &&
!blackBoxes[x-1][y]) { rotateTo(3); return; } //
West

// Priority 2: Try to move towards the target Y
coordinate
if(y < targetY && (y+1) <= 5 &&
!blackBoxes[x][y+1]) { rotateTo(0); return; } //
North
if(y > targetY && (y-1) >= 0 &&
!blackBoxes[x][y-1]) { rotateTo(2); return; } //
South

// Priority 3 (Avoidance): If the direct path is
blocked, find any valid move.
// Avoid turning back on itself unless necessary.
if((y+1) <= 5 && !blackBoxes[x][y+1] && direction
!= 2) { rotateTo(0); return; } // Try North
if((x+1) <= 5 && !blackBoxes[x+1][y] && direction
!= 3) { rotateTo(1); return; } // Try East
if((y-1) >= 0 && !blackBoxes[x][y-1] && direction
!= 0) { rotateTo(2); return; } // Try South
if((x-1) >= 0 && !blackBoxes[x-1][y] && direction
!= 1) { rotateTo(3); return; } // Try West

// If truly trapped, turn around.
rotateTo((direction + 2) % 4);
}
```

```
// ================== ULTRASONIC ==================
long readUltrasonic(){
digitalWrite(trigPin,LOW); delayMicroseconds(2);
digitalWrite(trigPin,HIGH); delayMicroseconds(10);
digitalWrite(trigPin,LOW);
long duration = pulseIn(echoPin,HIGH,4000); //
Using a faster timeout
long distance = duration*0.034/2;
return distance;
}

void advancePastIntersection() {
setMotor(120,120);
delay(300);
stopMotors(); // Stop after advancing to ensure a
stable sensor reading
}

//แสดงพิกัดปัจจุบัน (x, y) และทิศทาง (direction)
void printCurrentPosition(){
Serial.print("Current Position: (");
Serial.print(x); Serial.print(", ");
Serial.print(y);
Serial.print(") - Direction: ");
switch(direction){
 case 0: Serial.println("N"); break;
 case 1: Serial.println("E"); break;
```

```
 case 2: Serial.println("S"); break;
 case 3: Serial.println("W"); break;
 default: Serial.println("Unknown"); break;
}
}

void playsound(){
tone(buzzerPin,1000);
delay(250); // Shorter beep
noTone(buzzerPin);
delay(100);
}

void soundatposition(){
tone(buzzerPin,2000);
delay(250); // Shorter beep
noTone(buzzerPin);
delay(100);
}
```

refdontblow.ino

12 KB